# CO215, CAO LAB, Assignment 5

**Roll Number:** CSB24030

Name: HIMANSHU DIHINGIA

**Session:** Spring 2026

## Objectives

1. To get acquainted with reading integer input from user.
2. To understand and use indirect addressing mode.
3. To become familiar with loops and transfer of control instructions.

## Exercise 1: Program to Read Array of Single Digit Integers and Compute Sum

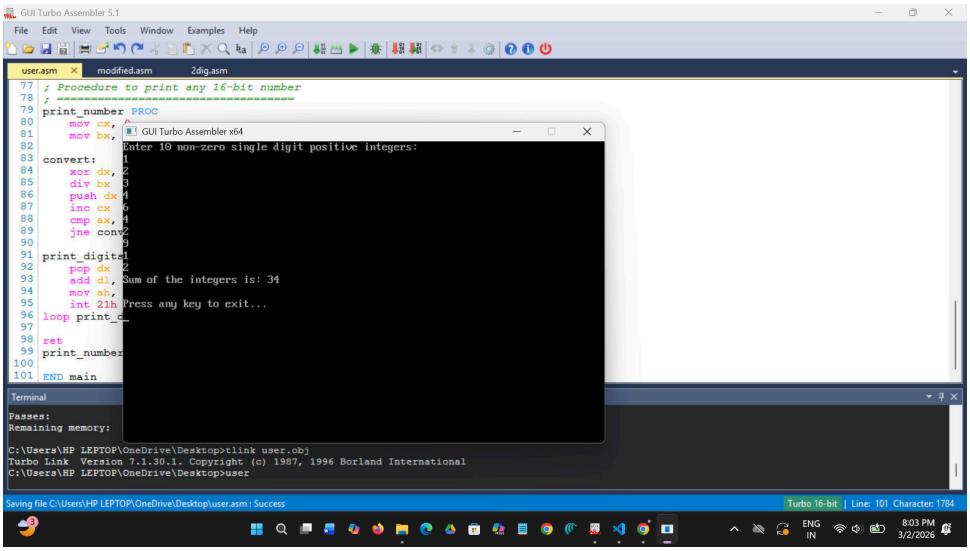
### Program Code

```
CR EQU 0DH
LF EQU 0AH

.MODEL SMALL
.STACK 100H

.DATA
array DW 10 DUP(?)          ; store 10 numbers
sum   DW 0

prompt_msg DB 'Enter 10 non-zero single digit positive integers
   :',CR,LF,'$'
output_msg DB CR,LF,'Sum of the integers is: $'

.CODE
main PROC
.STARTUP

; Display prompt
lea dx, prompt_msg
mov ah, 09h
int 21h

; ------------------------------
; Read 10 digits from keyboard
; ------------------------------
mov cx, 10
```

```
mov bx, OFFSET array

read_loop:
    mov ah, 01h          ; read character
    int 21h

    cmp al, 0DH          ; if ENTER pressed
    je read_loop         ; ignore and read again

    sub al, 30h          ; convert ASCII to number
    mov ah, 0
    mov [bx], ax         ; store number
    add bx, 2
loop read_loop

; ----------------------------
; Compute Sum
; ----------------------------
mov cx, 10
mov bx, OFFSET array
xor ax, ax               ; AX = 0

sum_loop:
    add ax, [bx]
    add bx, 2
loop sum_loop

mov sum, ax

; ----------------------------
; Display result message
; ----------------------------
lea dx, output_msg
mov ah, 09h
int 21h

; ----------------------------
; Display multi-digit number
; ----------------------------
mov ax, sum
call print_number

; Exit program
mov ah, 4Ch
mov al, 0
int 21h

main ENDP

; ===================================
; Procedure to print any 16-bit number
```

```
; ====================================
print_number PROC
    mov cx, 0
    mov bx, 10

convert:
    xor dx, dx
    div bx          ; AX / 10
    push dx         ; store remainder
    inc cx
    cmp ax, 0
    jne convert

print_digits:
    pop dx
    add dl, 30h
    mov ah, 02h
    int 21h
loop print_digits

ret
print_number ENDP

END main
```

## Output Screenshot




# Exercise 2: Assemble, Link and Execute

The program was successfully assembled, linked and executed. The correct sum was displayed after entering the integers.

# Exercise 3: Addressing Modes Used in the Program

The following addressing modes were used:

1. **Immediate Addressing Mode** Example: `mov ah, 09h`
2. **Register Addressing Mode** Example: `mov ax, sum`
3. **Direct Addressing Mode** Example: `mov sum, ax`
4. **Register Indirect Addressing Mode** Example: `mov [bx], ax`
5. **Indexed Addressing Mode** Example: `add ax, [bx]`
6. **Implied Addressing Mode** Example: `loop rd_loop`

# Exercise 4: Program to Read Any Number of Integers (Terminated by 0, Maximum 20)

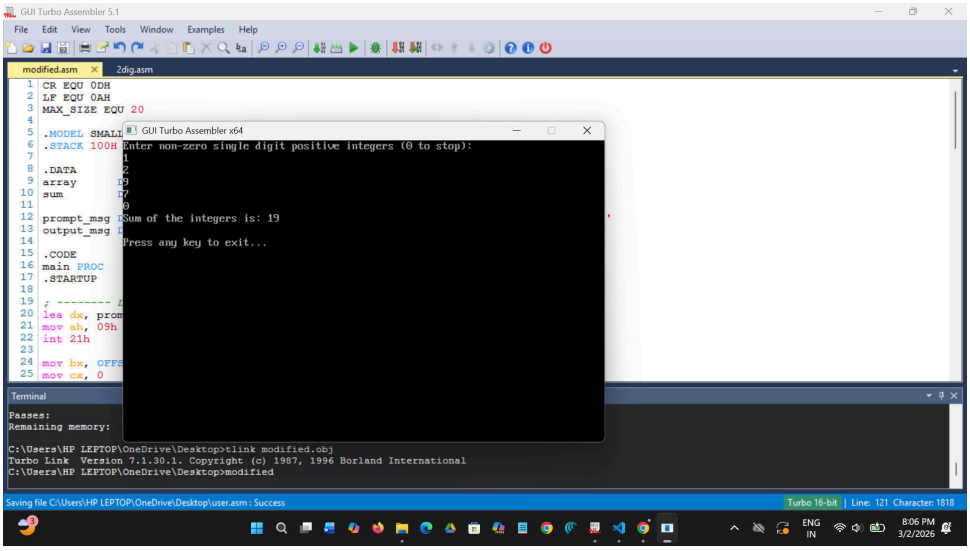
## Program Code

```
CR EQU 0DH
LF EQU 0AH
MAX_SIZE EQU 20

.MODEL SMALL
.STACK 100H

.DATA
array      DW MAX_SIZE DUP(?)
sum        DW 0

prompt_msg DB 'Enter non-zero single digit positive integers (0
   to stop):',CR,LF,'$'
output_msg DB CR,LF,'Sum of the integers is: $'

.CODE
main PROC
.STARTUP

; -------- Display Prompt --------
lea dx, prompt_msg
mov ah, 09h
int 21h

mov bx, OFFSET array
mov cx, 0                  ; counter = 0

; -------- Input Loop --------
rd_loop:
```

```
cmp cx, MAX_SIZE
je compute_sum

mov ah, 01h
int 21h             ; read key

cmp al, 0Dh         ; is it ENTER?
je rd_loop          ; ignore ENTER

cmp al, '0'
je compute_sum      ; stop if '0'

sub al, 30h         ; convert ASCII to number

mov ah, 0
mov [bx], ax

add bx, 2
inc cx
jmp rd_loop

; -------- Compute Sum --------
compute_sum:

mov bx, OFFSET array
mov ax, 0                   ; AX will hold sum

sum_loop:
cmp cx, 0
je display_result

add ax, [bx]
add bx, 2
dec cx
jmp sum_loop

; -------- Display Result --------
display_result:

mov sum, ax

lea dx, output_msg
mov ah, 09h
int 21h

; -------- Multi-digit Print Routine --------
mov ax, sum
call PRINT_NUM

; -------- Exit --------
mov ah, 4Ch
```

```
mov al, 00
int 21h

main ENDP

; ==========================================
; Procedure to print number in AX (any size)
; ==========================================

PRINT_NUM PROC
push ax
push bx
push cx
push dx

mov cx, 0           ; digit counter
mov bx, 10

convert_loop:
mov dx, 0
div bx              ; AX ? 10
push dx             ; store remainder
inc cx
cmp ax, 0
jne convert_loop

print_loop:
pop dx
add dl, 30h         ; convert to ASCII
mov ah, 02h
int 21h
loop print_loop

pop dx
pop cx
pop bx
pop ax
ret
PRINT_NUM ENDP

END main
```

## Output Screenshot




# Exercise 5: Program to Read Two Digit Integers

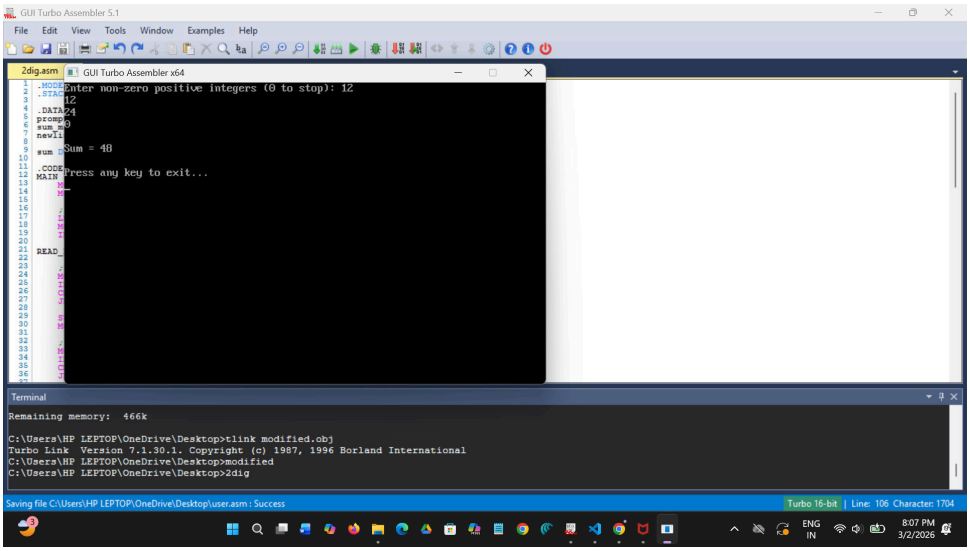
## Program Code

```
.MODEL SMALL
.STACK 100H

.DATA
prompt_msg DB 'Enter non-zero positive integers (0 to stop): $'
sum_msg    DB 0DH,0AH,'Sum = $'
newline    DB 0DH,0AH,'$'

sum DW 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Print prompt
    LEA DX, prompt_msg
    MOV AH, 09H
    INT 21H

READ_LOOP:

    ; Read first digit
    MOV AH, 01H
    INT 21H
    CMP AL, '0'
    JE CHECK_ZERO
```

```
    SUB AL, '0'
    MOV BL, AL          ; BL = first digit

    ; Read second digit
    MOV AH, 01H
    INT 21H
    CMP AL, 0DH         ; If Enter pressed after one digit
    JE SINGLE_DIGIT

    SUB AL, '0'
    MOV BH, AL          ; BH = second digit

    ; Convert to number = first*10 + second
    MOV AL, BL
    MOV AH, 0
    MOV CL, 10
    MUL CL              ; AX = first digit * 10
    ADD AL, BH          ; Add second digit
    JMP STORE_NUMBER

SINGLE_DIGIT:
    MOV AL, BL
    MOV AH, 0
    JMP STORE_NUMBER

CHECK_ZERO:
    ; If first digit is '0', check if next key is Enter
    MOV AH, 01H
    INT 21H
    CMP AL, 0DH
    JE DISPLAY_SUM
    JMP READ_LOOP

STORE_NUMBER:
    ; Add to sum
    MOV AH, 0
    ADD sum, AX

    ; Print newline
    LEA DX, newline
    MOV AH, 09H
    INT 21H

    JMP READ_LOOP

DISPLAY_SUM:

    ; Print sum message
    LEA DX, sum_msg
    MOV AH, 09H
    INT 21H
```

```
    ; Convert sum to decimal and print
    MOV AX, sum
    MOV BX, 10
    XOR CX, CX

CONVERT:
    XOR DX, DX
    DIV BX
    PUSH DX
    INC CX
    CMP AX, 0
    JNE CONVERT

PRINT:
    POP DX
    ADD DL, '0'
    MOV AH, 02H
    INT 21H
    LOOP PRINT

    ; Exit program
    MOV AH, 4CH
    INT 21H

MAIN ENDP
END MAIN
```

## Output Screenshot




# Learning Outcome

From this laboratory exercise, I learned:

- How keyboard input is taken using DOS interrupt 21H.
- How ASCII values are converted into numeric form.
- How indirect addressing works using BX as pointer.
- How LOOP and conditional jumps control program execution.
- How multi-digit numbers are handled in assembly.

This exercise strengthened my understanding of memory handling, addressing modes and control transfer instructions in 8086 programming.